

Bitcode

Measurement — Master Document

v2 · grounded in BITCODE_SPEC_V48 (measurement law, Gate 3, Gate 4) · explains and motivates, invents nothing · where this and SPEC diverge, SPEC wins · open tensions in §7.

The measurement theorem. No measurement, no price. No price, no settlement. No settlement, no market.

1 · Why measurement is the market

Every market that works has an instrument at its center. Grain has grades, gold has assays, securities have audited filings — and until now, code has had nothing but a demo and a promise. That is why code IP trades slowly, privately, and badly.

Bitcode's answer is to make measurement the instrument, and the SPEC is unusually direct about it: *measurement is the singular key to valuable IP commoditization and exchange; every sale or deposit decision must be grounded in measured readback*. What that grounding unlocks, concretely:

Prices without negotiation. The buyer's payment and the minted \$BTD both trace to measured readings — no nine-month licensing dance, no closed-door NDA pricing.

Comparability. Every DataPack carries the same full catalogue of absolute readings, so two packs can be compared the way **two bonds** can — row by row.

Search that sees structure. The Depository indexes measurements, not just text: static filters over absolute kinds, lexical match, and vector search — demand finds supply by what the code is, not what its README claims.

Trust before payment. Readers see Need coverage, fit confidence, provenance, novelty, risk, and the final BTD scalar before paying; depositors see their absolutes, demand posture, compensation expectation, and safety blockers before approving. Both sides act on readings, not vibes.

Honesty as a product surface. Every reading carries a status; every measure session carries a coverage report. The market can always tell measured from estimated from not-measured — which is precisely what today's data channels cannot offer.

2 · The object and its four measurement surfaces

The unit of commerce is the DataPack — always a fully synthesized artifact, never a raw source slice or a bare path list:

DataPack = commercial .patch + measurements + metadata

The patch is a plan descriptor plus a formal artifact with file bodies and unified diff (create/modify only — no delete ops). The metadata is commercially legible. And the measurements — the critical subsystem — arrive on one carrier with four surfaces:

Surface	When	What it is
absolutes	Deposit + read	Intrinsic properties; the full commercial catalogue, Σ weights = 1
needinesses	Read only	Reader/Need-relative *-fit dimensions
materialIdentity	Deposit + read + Exchange	Source-safe multi-valued identity: language mix, dependency inventories by usage, frameworks/services, tag sets, companion scalar volumes
measureReport	Deposit + read + Exchange	Measure-session honesty telemetry: bodies measured, coverage ratio, expanded-fill count, mode

Two laws frame everything. **Models do not invent measured values** — hosts and tools measure; agents reason over measured readings. And **deposit needinesses are always empty**: read-demand is never invented as a deposit

measurement.

3 · Absolutes — the pack, described

3.1 The catalogue

The absolute catalogue (*DATA_PACK_ABSOLUTES_CATALOG*) holds **65 kinds** whose weights sum to exactly 1 — one full catalogue, no smaller “weighted subset.” Its families: **structure, verification, hygiene, provenance, semantics, value**, plus material-identity companions. Representative examples (the catalogue package is SSOT for the full list):

Family slice	Examples	Authority
Classic structure	function-count, type-count, file-span, symbolic-richness, modularity, lang-span, test-surface, api-surface	Static-analysis report (report-owned)
Extended structure	dependency-span, doc-signal, config-surface, data-flow / control / connectivity	Static analysis + bare heuristics
Identity companions	language-concentration, framework-surface, dependency-class-balance, capability-surface, purpose-clarity	Material-identity extract → scalarVolumes
Quality estimates	correctness-estimate, objectives-fidelity, computational-usage	Optional measure-agent refine; never invented by synthesis LLMs

Hygiene kinds are first-class absolute packages; product policy may treat them as hard gates or penalties. *learning-gain* is explicitly *not* an absolute — exchange value is BTM via needinesses on read. And the load-bearing rule: **absolutes never mint BTM**. Their composite ($\sum \text{weight} \times \text{volume}$) is commercial legibility of supply, nothing more.

3.2 The reading, and the honesty statuses

Every reading is catalog-complete and carries its own honesty class:

```
{ measurementKind, label, weight, volume (0..1), magnitude, unit,
  category: 'absolute', status, descriptor?, rationale?, evidenceRoot? }
```

```
status = measured | estimated | insufficient_evidence
        | expanded-fill | not_run | not_implemented
```

The statuses are the trust architecture. A *measured* row came from evidence; an *estimated* row is a labeled heuristic; *expanded-fill* exists only so the bag is catalog-complete — volume 0, magnitude 0, and it may never be presented as a measurement. The honesty law is blunt: claiming volume 1 (“clean”) without bodies or a real scan is illegal; empty evidence yields *insufficient_evidence* or *not_run* at volume 0. The market never mistakes an unexamined pack for a clean one.

3.3 How measuring actually runs

Source set. `resolveMeasureSourceSet` assembles what tools may read: the pack's covered paths and file-change paths with bodies, measure-critical manifests (package.json, lockfiles, go.mod, Cargo.toml, pyproject.toml, ...) within obfuscation-allowed roots, optionally sibling tests — capped (default 80 bodies) with truncation telemetry, never silent, never outside the checkout/obfuscation allowlist. The measure unit is always the `DataPack`, after synthesis.

Measure stack. Static analysis + material-identity extract run first; bare per-kind measure packages fill their kinds; an optional quality PTRR refine may adjust quality estimates. Then the host merge law arbitrates: report-owned kinds (classic structure, dependency-span, doc-signal, config-surface) take the analyzer's magnitudes; material-identity scalarVolumes win for companion kinds; bare packages fill the rest — and an expanded-to-zero report row never overrides a real measure.

Telemetry. Every session emits a `measureReport` — bodies measured, path scope, coverage ratio, expanded-fill count, and a mode (deep when bodies ≥ 8 and coverage ≥ 0.5 , thin when any bodies, else path-only). Fail-soft thin measure is always preferred over silent zeros.

Who measures, when (deposit). Discovery's `comprehend-codebase` measures the checkout to ground the knowledge map; the Implementation measurement agent runs the stack and attaches measurements: { `absolutes`, `materialIdentity?`, `measureReport?` }; Validation is fail-closed — no catalog-complete absolutes with magnitude +

volume, no finish — and must not re-measure. LLM agent JSON must never invent absolute, neediness, or identity volumes. Read Implementation shares the same deep source-set and honesty law.

Prompt discipline. Each inference-owned measurement is commanded by named Prompt/PromptPart composition, recording context class, source boundary, template identity, source-safe digest, typed schema, parsed result, proof root, telemetry receipt, and repair posture. Raw prompts and unpaid source stay private; identity, digest, typed result, and proof root are disclosable — measurements come with receipts.

3.4 What the market sees

Deposit option cards show the measure-report strip, material identity with dependencies-by-usage, and an honesty badge per absolute — fill rows say “Not measured — catalogue placeholder.” Exchange detail shows the same, source-safe. Depositors get a review artifact (path-op patch + metadata + absolutes with status + identity + measureReport + honesty counts, no unpaid bodies). Search filters on absolute kinds. Measurement is not a back-office step; it is the product's face.

4 · Needinesses — the pack, scored against a Need

Needinesses exist only at read, only as *-fit kinds, under the same numeric law (magnitude + volume, category 'neediness'). A read that arrives with empty needinesses, or any kind not ending in -fit, fails closed.

Static catalogue. Fixed dimensions with fixed weights — e.g. language-fit, domain-fit, interface-fit (ASSET_PACK_NEEDINESSES_CATALOG; the package is SSOT for the exact list and weights — implementation currently carries 0.35 / 0.35 / 0.30).

Dynamic rows. Setup's comprehend-needs plans additional dimensions from the specific Need, grounded in checkout paths and topics, as first-class plan rows:

dynamicNeedinesses: [{ measurementKind (MUST end -fit), label, guidance, weight }]

Implementation host-attaches measured volumes for static + dynamic rows onto each option — the same “hosts measure, models don't invent” law as absolutes.

The mass law. When both families are present, static carries 0.6 of the weight mass and dynamic 0.4, each family re-normalized internally so all neediness weights sum to 1 before compositing. A Need can extend the ruler; it cannot replace it.

The composite. need-fit is never a raw row an agent targets — it is derived:

needFitVolume = **weightedMean(needinesses[].volume)** ∈ [0,1]

using each row's weight (reading weight, else catalogue weight, else equal). Final read options must carry the full measurements bag plus needFit.

This is the market-empowerment heart of the design: the buyer states what they need, and the exchange measures *that* — fit to this demand, on this day — rather than asking the buyer to trust a generic score. **Demand itself becomes a measuring instrument.**

5 · From measurement to money

Read produces multiple options; the buyer selects; each bought option starts its own settle pipeline (1 : 1, never batched). Buyers never pay in BTD — they pay ETH, BTC, or SOL at spot vs BTD (mock spots on testnet). The stages, per SPEC: validate readiness (exactly one option) → observe payment → quote → finalize → ship the patch as a PR on the read repo → journal to /exchange.

The quote is where measurement becomes money, and only needinesses participate:

needFitVolume = $\sum (w_i \times \text{clamp01}(\text{volume}_i)) / \sum w_i$ ∈ [0,1]

rawV = **floor(needFitVolume × 10¹⁸)** ≤ 1 whole BTD per settle

V = **floor(rawV × decay)** decay from residual 21M supply

\$BTD is the finite fungible token (BitcodeERC1155 id 0, cap 21,000,000, 18 decimals); DataPack co-ownership is ids ≥ 1 , add-only, burn forbidden. The buyer's receipt is the co-own NFT plus delivery — **buyers never receive minted BTD**. Depositor economics split V by source-to-shares and per-depositor btdBps/coinBps (the coin leg takes the protocol fee; the BTD leg has none), with the SPEC's pay/earn law stating that **unchosen BTD is not minted**. After settle, the seller finalizes sellerBtdBps on /exchange — their BTD-vs-ETH split of the escrowed economics. (On the escrow-mint vs leftover-mint tension inside the SPEC, see §7-U1 — this document does not resolve it.)

Deposit-side BTD, by contrast, is only ever an **estimated, potential range** — no reviewed buyer Need exists at deposit, so nothing mints. Depositors see legibility; readers make markets.

6 • The honesty machinery, in one place

What makes a Bitcode measurement worth trusting is not any single reading — it is that every layer refuses to overstate:

- 1 • Statuses on every absolute.** Measured vs estimated vs not-measured is always visible, badged in the UI, carried into artifacts and projections.
- 2 • measureReport on every session.** Coverage and mode published, expanded-fill counted, truncation telemetered.
- 3 • Models never invent volumes.** Synthesis LLMs cannot emit measured values; hosts attach; validation fail-closes without them and never re-measures.
- 4 • No fake clean.** Hygiene volume 1 without evidence is illegal by law of the catalogue.
- 5 • Receipts.** Named prompts, digests, typed schemas, proof roots on every inference-owned measurement; disputes replay rather than argue.
- 6 • The mint is fenced.** Absolutes never mint; needinesses only, composite-only, clamped, weighted, decayed, fail-closed on emptiness.

7 • Uncertainties to resolve (flagged, not resolved here)

Genuine open points, either internal to SPEC V48 or between SPEC and implementation. Nothing below is guessed at in the body of this document.

U1 — Escrow-mint vs leftover-mint. G4-5 stage 4 mints 100% of V to escrow, and seller-payout finalize routes the inverse remainder of each asset to the treasury (e.g. sellerBtdBps=1000 → seller 10% BTD + 90% ETH; treasury 90% BTD + 10% ETH). The same section's pay/earn law states “unchosen BTD is not minted.” The canonical tokenomics document is leftover-mint (declined BTD never created). These are different supply mechanics — under escrow-mint the treasury accumulates unelected BTD; under leftover-mint it never exists. Which is V0 law and which is the target needs a SPEC ruling, and the tokenomics doc must follow it.

U2 — Fee shape. SPEC: the coin leg takes the protocol fee, the BTD leg has none. Canonical tokenomics: a dynamic fee on both buckets under the self-sustainability invariant. Reconcile (asymmetric fixed fee vs symmetric dynamic fee), and place the sustainability invariant in SPEC if it is law.

U3 — Two elections, one seller? btdBps/coinBps per depositor share (mint destination) and sellerBtdBps (escrow payout finalize) both exist. Their relationship — who is “depositor” vs “seller” in a multi-contributor pack, and whether both elections apply to the same economics — deserves one worked example in SPEC.

U4 — Neediness weight fallbacks. SPEC's mass law (0.6 static / 0.4 dynamic, re-normalized) supersedes the implementation's earlier “unknown kind → weight 1” fallback; parity should be verified so no path still admits unweighted unknown kinds into the mean.

U5 — Deposit-side BTD estimate. The SPEC declares it an estimated/potential range but does not specify its computation. Unspecified — do not present deposit estimates as if produced by a defined formula.

U6 — confidence and riskAdjustment. The weighted-scalar rule multiplies volume × confidence × riskAdjustment × weight “where applicable,” while the need-fit composite uses weight × clamped volume. Which kinds carry the extra factors, and where they enter, is not fully specified.

U7 — Fit-scoring stability. The measurement prompt rule fixes identity and receipts, but nothing yet pins judge/model versions per epoch or defines golden Need↔pack fixtures — the anchoring that keeps needFit = 0.76 meaning the same thing across months. Still open.

Advanced Engineered Software, Inc. · v2, SPEC-V48-grounded, July 27, 2026 · This document explains the law and its market consequences; it is not the law · Catalogue packages and the parity matrix are SSOT for kind lists and weights · Resolve §7 through the SPEC gate.